Lecture compiled defunctionalization of Bintree to flat array. Org/Visualization

- green: processed (value is in the acc)
- yellow: currently being looked at
- grey: unprocced.

callstack: The JS engine's stack (grows down)

continuation chain: The nested DoRight objects.

step 1: The initial call

flatten_defunc(tree, [], makeDone())

1. Evaluate makeDone() → returns {tag: 0}
2. Call flatten_defunc(tree, [], {tag: 0})

call stack:

flatten_defunc(node, [], Done),

main call:
flatten_defunc(tree[1], makeDone())

- makeDone() returns {tag:0}
- flatten_defunc(node4, [], {tag:0}) is called.

call 1: flatten_defunc(node4, [], {tag:0})

- t = node4, acc = [], cont = {tag:0}.
- t.tag is Tree.N
- we call: flatten_defunc(t.l, acc, makeDoRight(t.v, t.r, cont))

- makeDoRight(4, node6, {tag:0}) creates:

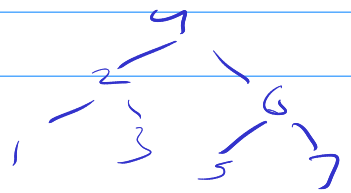
{tag:1, v:4, r:node6, cont:{tag:0}}

- chain is now: [DoRight(4)].

call 2: flatten_defunc(node2, [], DoRight(4))
t = node2, acc = [], cont = DoRight(4).
t.tag is Tree.N.

- we call makeDoRight(2, node3, cont)
- makeDoRight(2, node3, DoRight(4)) creates:
{tag:1, v:2, r:node3, cont:{tag:1, v:4, r:node6, cont:{tag:0}}}
- chain is now: [DoRight(2)] nested over [DoRight(4)]

call 3: flatten_defunc(node1, [], DoRight(2))



t = node1, acc = [], cont = DoRight(2).

tag is Tree.N

we call makeDoRight(1, leaf, cont)

makeDoRight(1, leaf, DoRight(2)) creates:

{tag: 1, v: 5, r: leaf, cont: {tag: 1, v: 2, v: node3, cont: {tag: 1, v: 4, r: node6, cont: {tag: 0}}}}

Chain is now: [DoRight(1)] over [DoRight(2)] over [DoRight(4)]

Call4: flatten_defunc(leaf, [], DoRight(1))

t = leaf, acc = [], cont = DoRight(1).

t.tag is tree.L

- we call apply(DoRight(1), [])